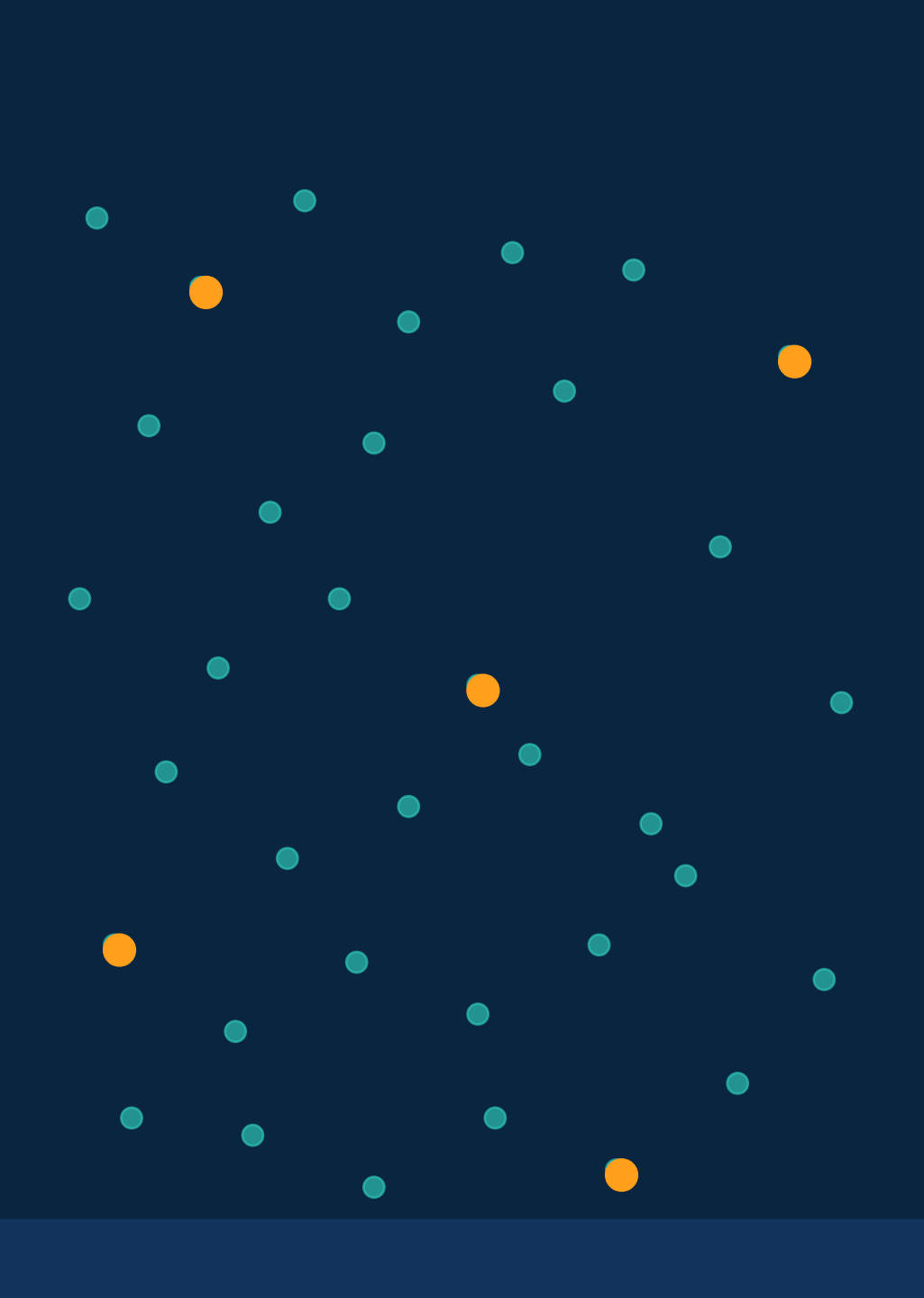




# PointNet

## *Deep Learning on Point Sets*

A complete technical walkthrough — architecture, classification, segmentation, and the road to PointNet++

# Modules of this Presentation

01

## The 3D Data Problem

Why deep learning struggled with 3D before PointNet.

03

## Earlier Alternatives

Voxels, meshes, sorting tricks, and RNN-based ideas.

05

## PointNet Architecture

The end-to-end network as proposed in the paper.

07

## Segmentation Network

Local + global features and the 1088-dim trick.

02

## Point Clouds Explained

What a point cloud is and how it differs from images.

04

## Properties of Point Sets

Unordered. Local interaction. Transformation invariance.

06

## Classification Pipeline

Module-by-module walkthrough of the algorithm.

08

## Weaknesses & PointNet++

What PointNet misses and why PointNet++ exists.

# Why PointNet Was Needed

*Deep learning was thriving on regular data — but 3D was a wall.*

## Images

CNNs

Grid of pixels — fixed neighborhood.

## Text

RNNs / Transformers

Sequential, ordered tokens.

## Audio

1-D conv / sequence models

Regularly sampled waveforms.

## But 3D point clouds broke every assumption

**Unordered** — rows can be shuffled and the object is still the same.

**Irregular** — no grid, no fixed neighbors, no canonical layout.

**Variable size** — a scan can return 1k points or 100k, all valid.

**Invariant to rigid motion** — rotation and translation must not change the label.

# What is a Point Cloud?

## Definition

A point cloud is a set of points sampled from the surface of a 3D object — typically captured by LiDAR, depth cameras, or photogrammetry.

### Mathematical form

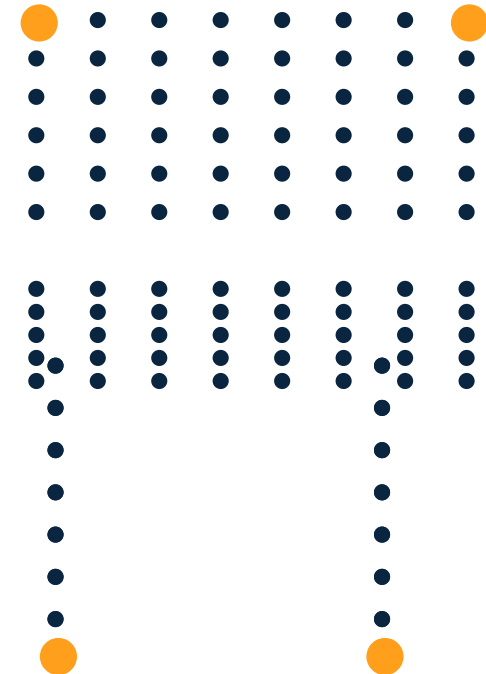
$S = \{ p_1, p_2, p_3, \dots, p_n \}$  where  $p_i = (x, y, z)$

### Stored as an $N \times 3$ matrix

$$\begin{bmatrix} 0.2 & 0.5 & 1.0 \\ -0.1 & 0.7 & 0.3 \\ 0.9 & -0.2 & 0.8 \\ \vdots & \vdots & \vdots \end{bmatrix}$$

Each row = a single point. Columns = (x, y, z).

## Example • LiDAR scan of a chair



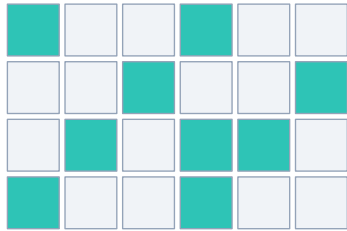
Thousands of (x,y,z) samples — no grid, no ordering.

# Alternatives Before PointNet · Part 1

*Researchers tried to force point clouds into something CNNs could handle.*

## Voxelization

Treat 3D space as a grid of cubes (like Minecraft) and run a 3-D CNN.

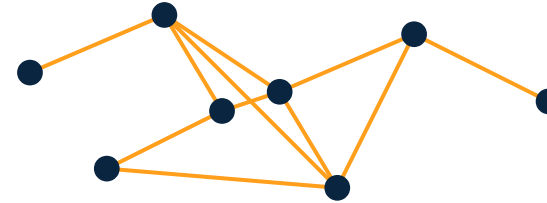


### Drawbacks

- Memory blows up cubically with resolution
- Most cells are empty — wasted compute
- Fine surface detail is lost to quantization

## Polygon Meshes

Connect points into triangles or polygons and reason over surfaces.



### Drawbacks

- Topology is irregular — hard for CNN-style kernels
- Mesh quality depends on reconstruction pipeline
- Connectivity must be computed up-front; not raw data

# Alternatives Before PointNet · Part 2

*Other attempts tried to keep points raw — and ran into the ordering problem.*

## Sorting the Points

Sort the  $N$  points by some rule (e.g.,  $x$  then  $y$  then  $z$ ), then feed them into a standard network.

### Why it fails

- No total order on  $\mathbb{R}^3$  is stable under perturbation — a tiny noise can flip the sort
- The network would have to learn the sorting rule, which defeats the purpose
- Empirically, sorted MLPs perform poorly compared to permutation-invariant designs

## Recurrent Networks (RNNs)

Treat the point cloud as a sequence and process it with an LSTM/GRU, hoping order won't matter.

### Why it fails

- RNNs are inherently order-sensitive — different shuffles give different outputs
- Difficult to scale to thousands of points per object
- Long-range dependencies across points are weakly modeled

# Three Properties of Point Sets

*Any network that processes raw point sets must respect these three properties.*

## 01 Unordered

### Permutation Invariance

A point cloud is a SET, not a sequence. Shuffling the rows of the  $N \times 3$  matrix does not change the underlying object, so the network output must be identical under any permutation.

$$f(x_1, x_2, \dots, x_n) = f(x_{\pi(1)}, x_{\pi(2)}, \dots, x_{\pi(n)})$$

## 02 Interaction among points

### Local Structure

Points are not isolated — neighbors form meaningful local geometry (edges, surfaces, corners). The model should be able to combine information from nearby points to capture shape.

$$\text{neighborhood}(p_i) \rightarrow \text{local descriptor}$$

## 03 Invariance under transformation

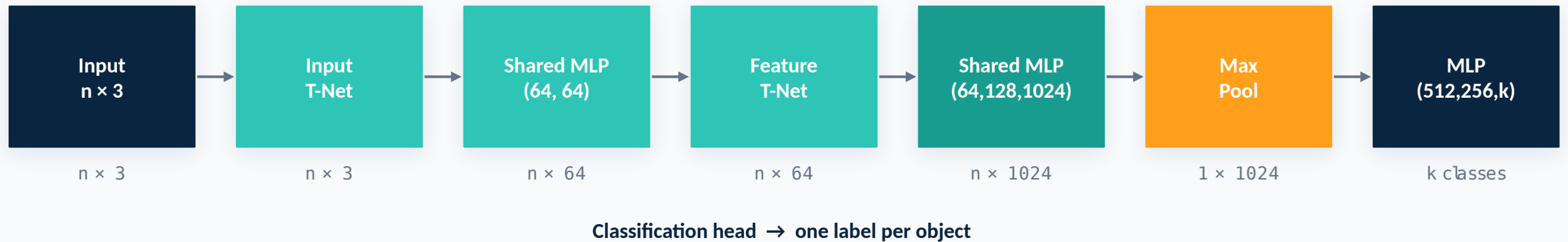
### Rigid Motion

Rotating or translating the whole object does not change what it is. A rotated chair is still a chair, so labels must remain stable under rigid transformations applied to all points jointly.

$$f(R \cdot X + t) = f(X) \quad \text{for rotation } R \text{ and translation } t$$

# The PointNet Architecture

*End-to-end pipeline shared by classification and segmentation.*



## Segmentation branch

Take per-point features ( $n \times 64$ ) and concatenate with the repeated global feature ( $n \times 1024$ ) → ( $n \times 1088$ ). Feed through another shared MLP to produce a label for every point.

**Output →  $n \times m$  (per-point class scores)**

# Input Transform — The First T-Net

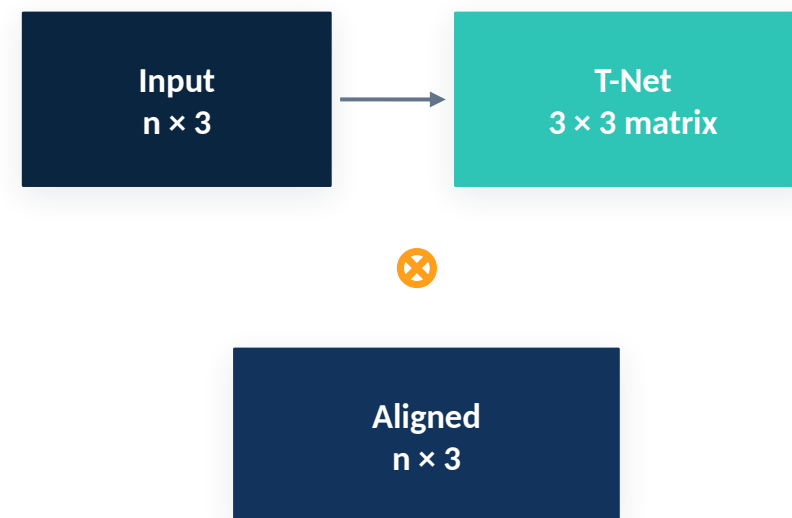
## Goal — align the input coordinates

Objects can arrive in any orientation. T-Net learns a small  $3 \times 3$  transformation matrix that rotates and scales the point cloud into a canonical pose before feature extraction.

### How it works

- T-Net is itself a mini-PointNet: shared MLP  $\rightarrow$  max-pool  $\rightarrow$  fully-connected layers
- It outputs the 9 entries of a  $3 \times 3$  matrix
- Each input point is multiplied by this matrix:  $x' = T \cdot x$
- Like auto-aligning every chair to face forward before classification

### Tensor flow



*Result — rotation-aligned point cloud,  
ready for feature extraction.*

# Shared MLP — Per-Point Feature Lift

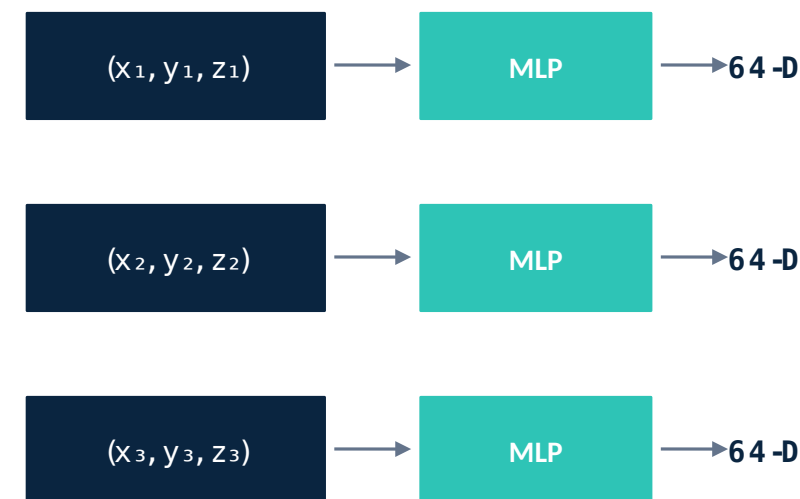
## Goal — lift each point into a 64-D feature

Every point is passed independently through the same small neural network:  $3 \rightarrow 64 \rightarrow 64$ . The SAME weights are reused for all  $N$  points — this is what 'shared' means.

### Why share the weights?

- Order-agnostic — every point gets the same operator
- Equivalent to a  $1 \times 1$  convolution over the  $N \times C$  tensor
- Drastically fewer parameters than a dense  $N$ -input layer
- Each point now carries a learned local descriptor

### Shared weights



*Same weights applied independently to every point  $\rightarrow n \times 64$*

# Feature Transform — Aligning the 64-D Space

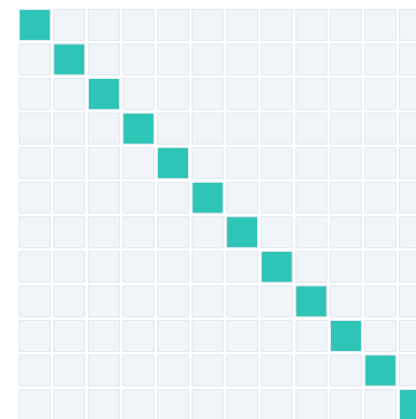
## A second T-Net — for features, not coordinates

Once each point has a 64-D feature, PointNet applies another T-Net to align the FEATURE space. This time the learned matrix is  $64 \times 64$ .

### Why again?

- Even after coordinate alignment, learned features can still vary
- A  $64 \times 64$  transform realigns the feature space across instances
- Large  $64 \times 64$  matrices are hard to optimize → a regularization term keeps the matrix close to orthogonal
- Improves stability and modestly improves accuracy

### Idea



*Learned  $64 \times 64$  feature-space matrix*

**Regularizer:**  $\| \mathbf{I} - \mathbf{A} \cdot \mathbf{A}^T \|^2$

Keeps the transform close to an orthogonal matrix so features don't collapse.

# Deep Shared MLP — 64 → 128 → 1024

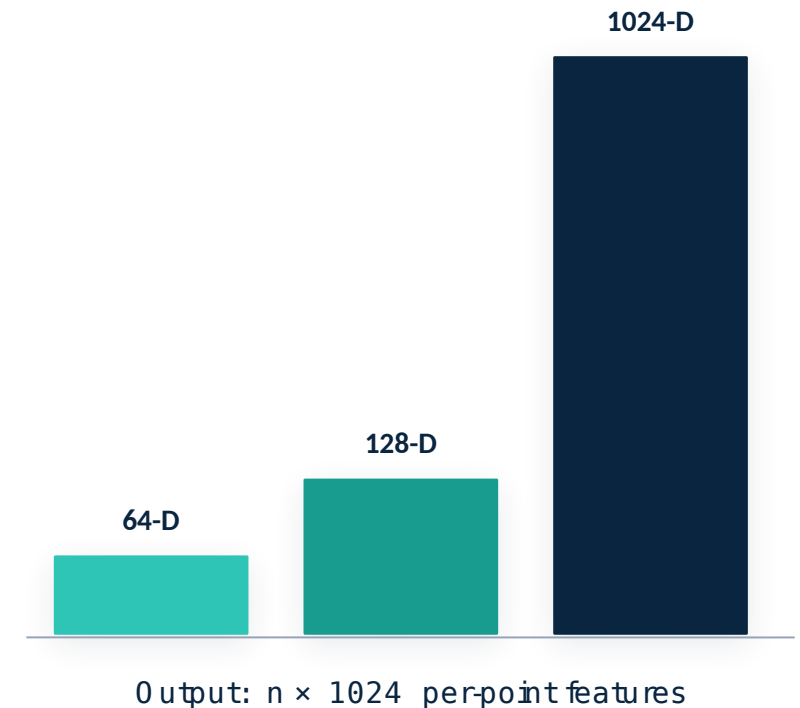
## Growing each point's descriptor

Each aligned 64-D point feature is passed through another shared MLP — 64 → 128 → 1024 — independently per point.

### What gets learned along the way

- Early layers — simple geometric primitives (edges, planar patches)
- Middle layers — curves, corners, surface motifs
- Deeper layers — semantic patterns: 'leg-like', 'flat-surface-like'
- **At this point, each point still has NO knowledge of the others**

Per-point feature dimension grows



# Max Pooling — The Key Insight

**Combining N points into ONE shape vector — without caring about order.**

For every one of the 1024 feature columns, take the maximum value across all N points. Result: a single  $1 \times 1024$  GLOBAL feature vector.

## Why this is permutation-invariant

$$\text{max}(1, 5, 2) = 5$$

$$\text{max}(2, 1, 5) = 5$$

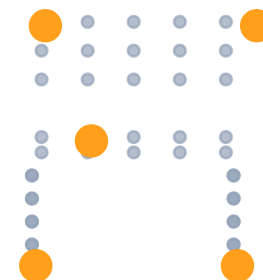
The max operator is symmetric — shuffling its inputs cannot change its output. That is exactly the property we need for an unordered point set.

$$f(x_1, \dots, x_n) = \gamma \circ \text{MAX}\{h(x_1), \dots, h(x_n)\}$$

*The general form proved in the paper — any symmetric function on point sets can be approximated this way.*

## Critical Points

Only a SPARSE subset of points 'wins' the max in each feature column. These winners — the critical points — form the learned shape skeleton.



Highlighted points (orange) dominate the max-pool

# Classification Head & Output

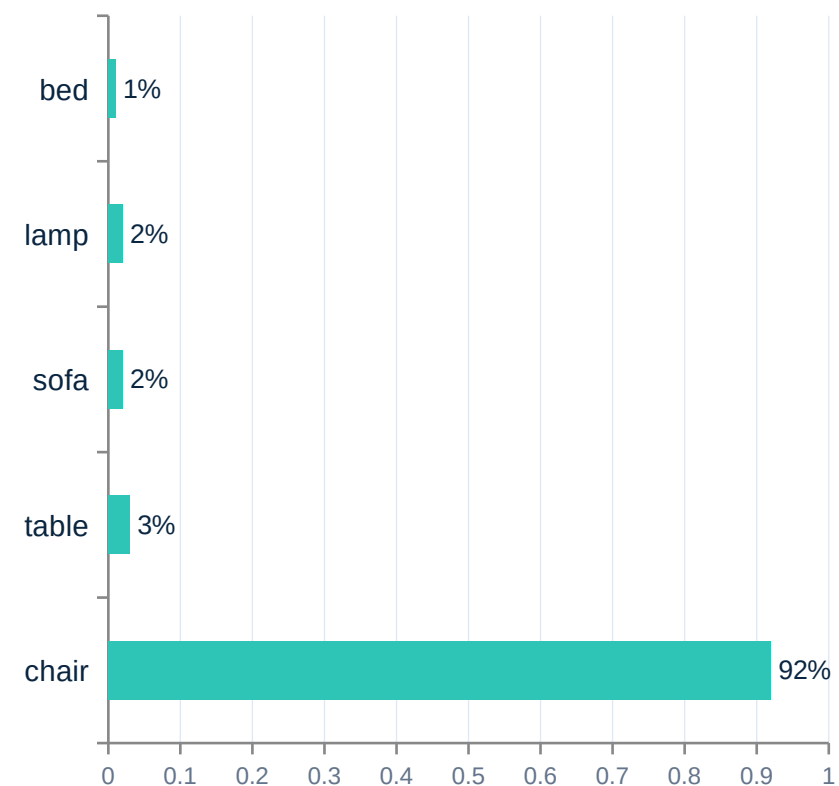
## From global feature to class label

The  $1 \times 1024$  global feature vector is fed into a fully-connected classifier:  $1024 \rightarrow 512 \rightarrow 256 \rightarrow k$ , where  $k$  is the number of object classes.

### Output

- A softmax over  $k$  classes: one score per category
- Training loss is standard cross-entropy
- Plus the T-Net orthogonality regularizer
- Example: ModelNet40 has  $k = 40$  (chair, table, lamp, sofa, ...)

### Example output • softmax



# From One Label to Per-Point Labels

*Classification answers 'what is this object?' · Segmentation answers 'what is each point?'*

## Classification

Input · point cloud

Output · ONE class label

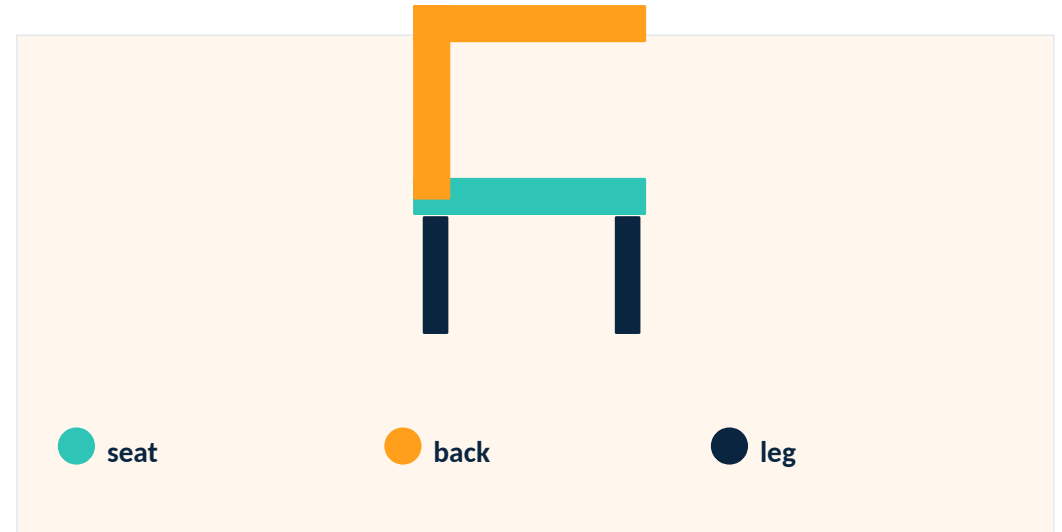


*ONE label · scope = entire object*

## Segmentation

Input · point cloud

Output · a label for EACH point

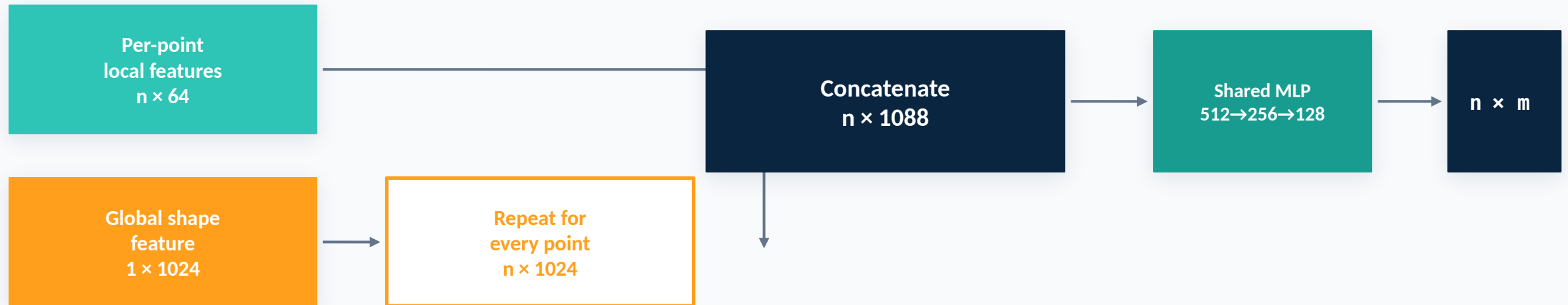


*n labels · scope = each point individually*

# Local + Global Fusion · the 1088-dim Trick

*Why max-pool alone is not enough*

After max-pooling, the network has a single  $1 \times 1024$  shape vector — it knows 'this is a chair', but has lost per-point identity. Segmentation needs both.



**Why 1088?  $64 \text{ (local)} + 1024 \text{ (global)} = 1088$**

Every point now answers two questions at once — 'What do I locally look like?' and 'What overall object am I part of?' — letting the MLP assign part labels (seat, back, leg, ...) with full context.

# Weaknesses & the Need for PointNet++

## Weaknesses

PointNet processes each point independently — only combining them with a single, global max-pool. This is also its biggest blind spot.

### No local neighborhood structure

Nearby points are never explicitly grouped or compared.

### Misses fine-grained local geometry

Small surface patterns, curvature, and corners get blurred.

### Single global representation

Hierarchical patterns at multiple scales are not captured.

## PointNet++ · the fix

Apply PointNet recursively on local neighborhoods — building a hierarchy similar to a CNN over images.

### Sampling layer

Pick a subset of representative centroids via farthest-point sampling.

### Grouping layer

Form local regions using ball-query or k-NN around each centroid.

### PointNet on each region

Extract local features → repeat at coarser scales → multi-scale hierarchy.

## C R E D I T S

*Based on the original research paper*

---

# PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation

## A u t h o r s

Charles R. Qi · Hao Su · Kaichun Mo · Leonidas J. Guibas

## I n s t i t u t i o n

Stanford University

## C o n f e r e n c e

*CVPR 2017 — IEEE Conference on Computer Vision and Pattern Recognition*